Stock Portfolio Management & Analytics System

> Database Systems Lab (CSS 2212)

Harshit Pandya	Prasiddhi Singhal	Samarth Rastogi
[240968444]	[240968448]	[240968494]

Manipal Institute of Technology | April 2026

A Full-Stack Database-Driven Engine

MySQL 8.0+ Core

The absolute source of truth. All business logic, risk metric computation, and portfolio updates are encapsulated in stored procedures and triggers.

Java REST API

A lightweight, framework-free com.sun.net.httpserver routing layer. Connects via JDBC to execute DB commands and serialize JSON.

Vanilla JS Dashboard

A responsive, single-page application (SPA) styled with a dark themes aesthetic. Consumes endpoints asynchronously.

Bridging the Gap in Retail Portfolio Analytics

The Gap

Manual Spreadsheets:
Prone to data inconsistency and entry errors.

Third-Party Platforms:
Expensive subscriptions locked behind proprietary walled gardens.

Missing Metrics:
Basic apps lack institutional-grade quantitative risk analysis.

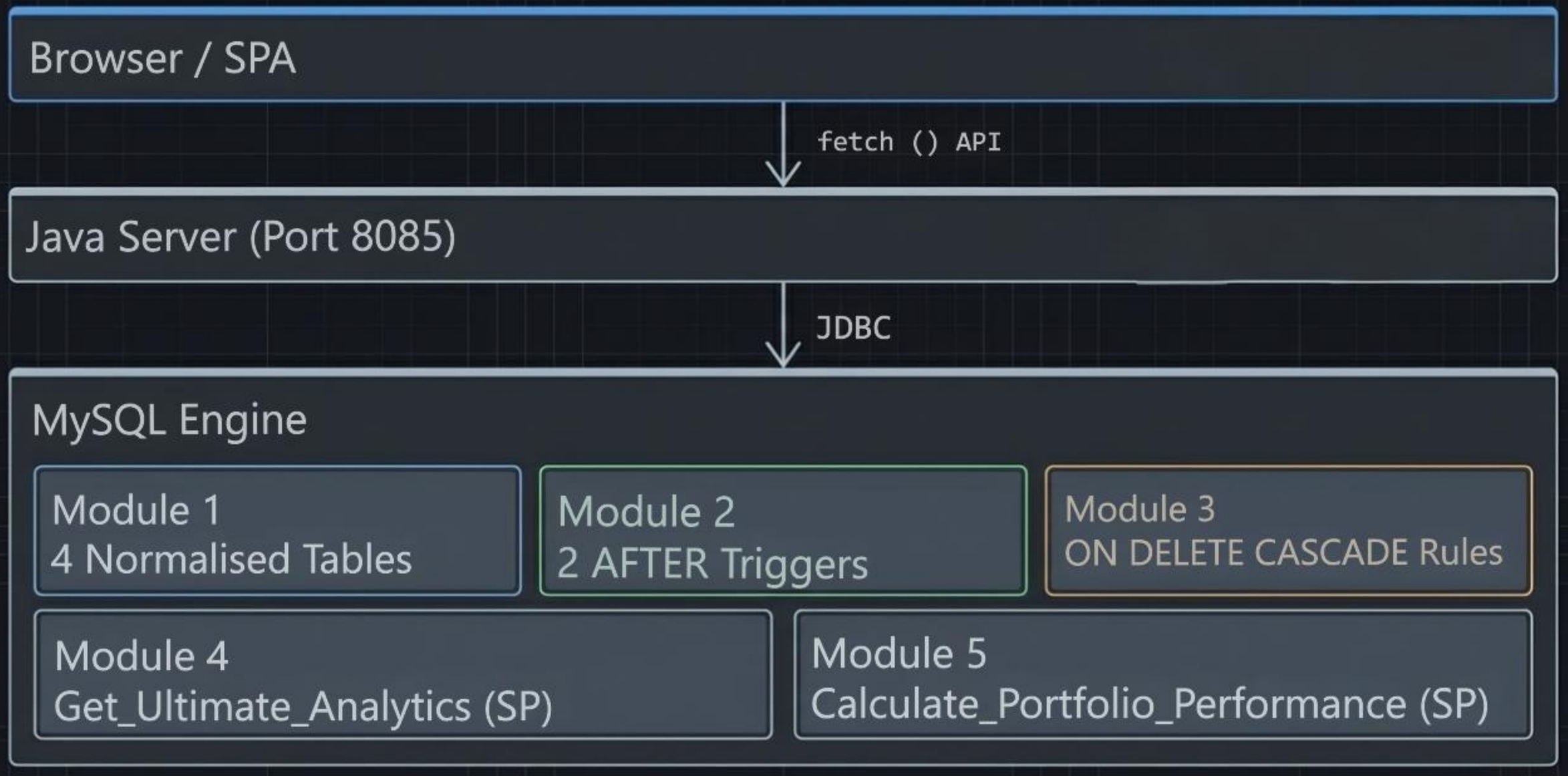
The System

Schema-First Design:
Database enforces 100% data integrity automatically.

Self-Hosted Engine:
Complete ownership of data without recurring costs.

Database-Native Analytics:
Complex calculations (Turnover Velocity, Concentration Risk) execute instantly at the storage layer.

Three-Tier Schema-First Architecture



Entity-Relationship Modeling

1:M Architecture

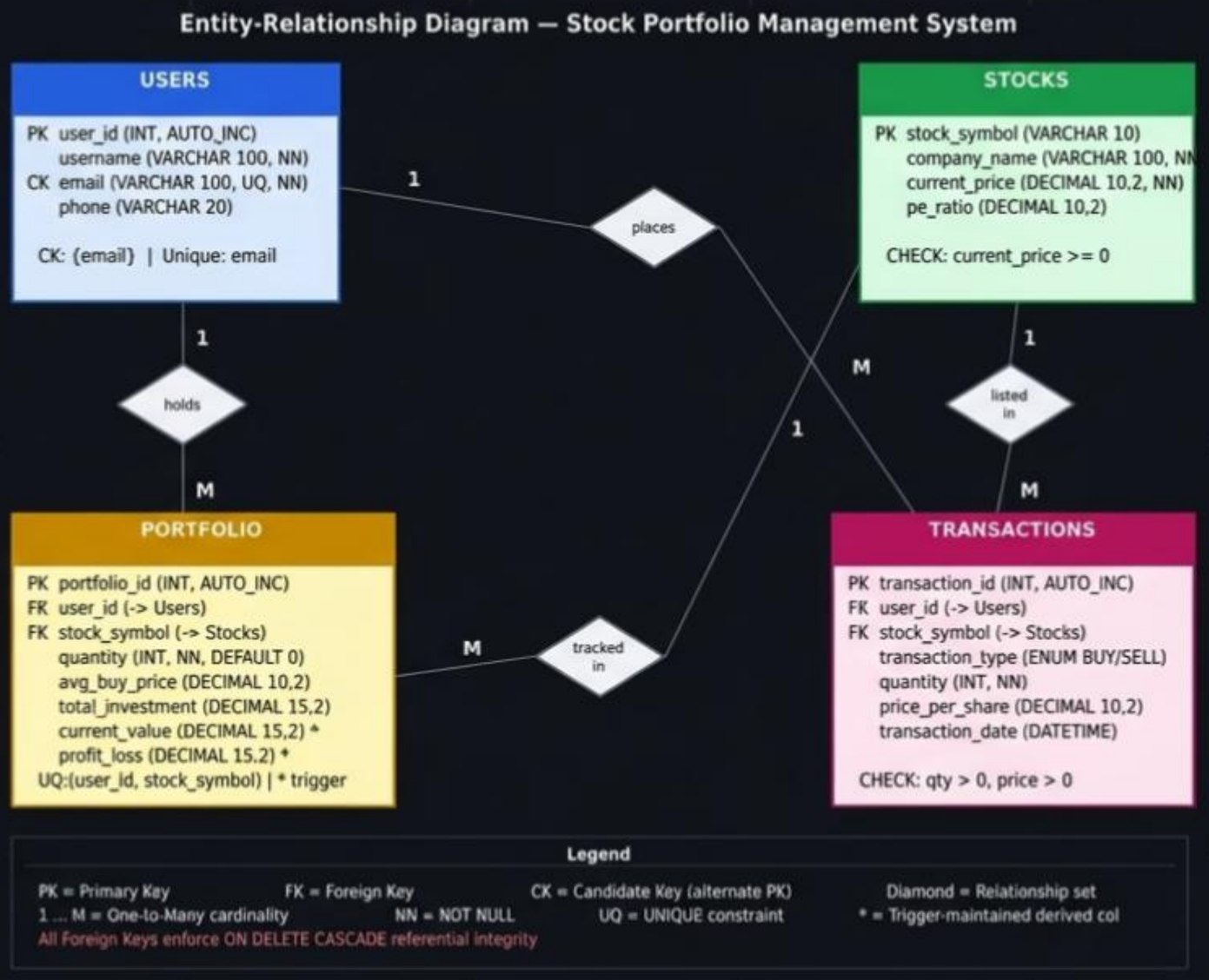
Four core entities (Users, Stocks, Portfolio, Transactions).

Referential Integrity

All foreign keys enforce ON DELETE CASCADE.

Composite Guard

The Candidate Key {user_id, stock_symbol} guarantees one Portfolio entry per stock per user.



Schema Normalisation Diagnostic Matrix

Table	1NF	2NF	3NF	BCNF
Users	[✓]	[✓]	[✓]	[✓]
Stocks	[✓]	[✓]	[✓]	[✓]
Transactions	[✓]	[✓]	[✓]	[✓]
Portfolio	[✓]	[✓]	[*]	[✓]

The 3NF Exception (Controlled Denormalisation)

`current_value` and `profit_loss` are deliberately cached in the `Portfolio` table. They are derived from `quantity` and `Stocks.current_price`. This avoids costly real-time joins on massive dashboards, maintained strictly by triggers.

DDL & Embedded Integrity Constraints

`AUTO_INCREMENT`

Surrogate primary keys on Users, Portfolio, and Transactions.

`CHECK (price >= 0)`

Domain constraint preventing negative stock prices or trade quantities.

`UNIQUE (email)`

Enforces business logic; serves as a candidate key for Users.

`ENUM ('BUY', 'SELL')`

Hard-coded transaction type restriction preventing invalid API inputs.

`ON DELETE CASCADE`

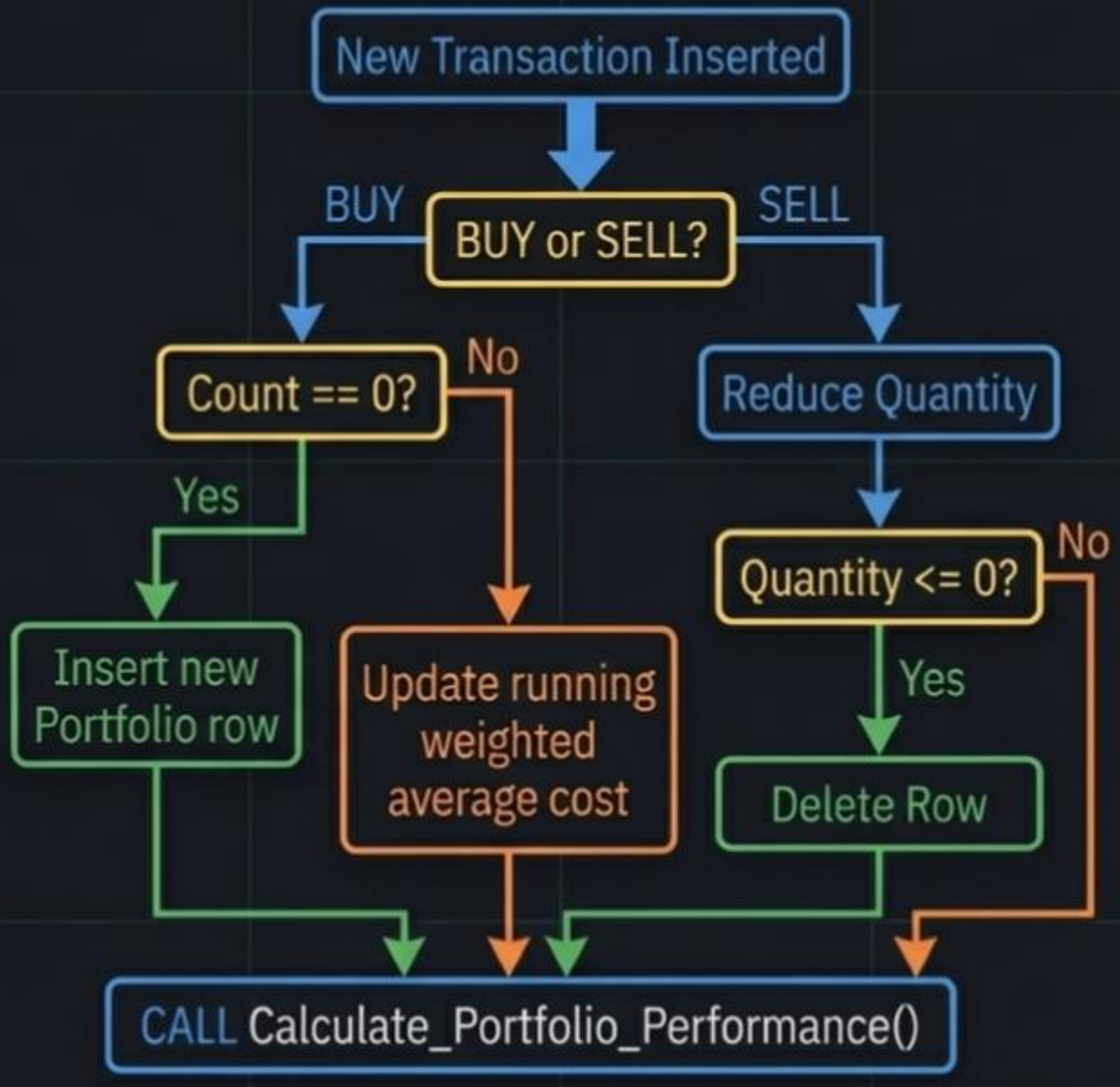
Prevents orphaned portfolio or transaction records if a User or Stock is removed.

Transaction Automation: State Machine

```
mySQL>

CREATE TRIGGER After_Transaction_Insert
AFTER INSERT ON Transactions
FOR EACH ROW
BEGIN
  -- Logic to handle BUY and SELL transactions
  -- and update Portfolio state

  IF NEW.transaction_type = 'BUY' THEN
    ...
  END IF;
  ELSE IF NEW.transaction_type = 'SELL' THEN
    ...
  END IF;
END IF;
```

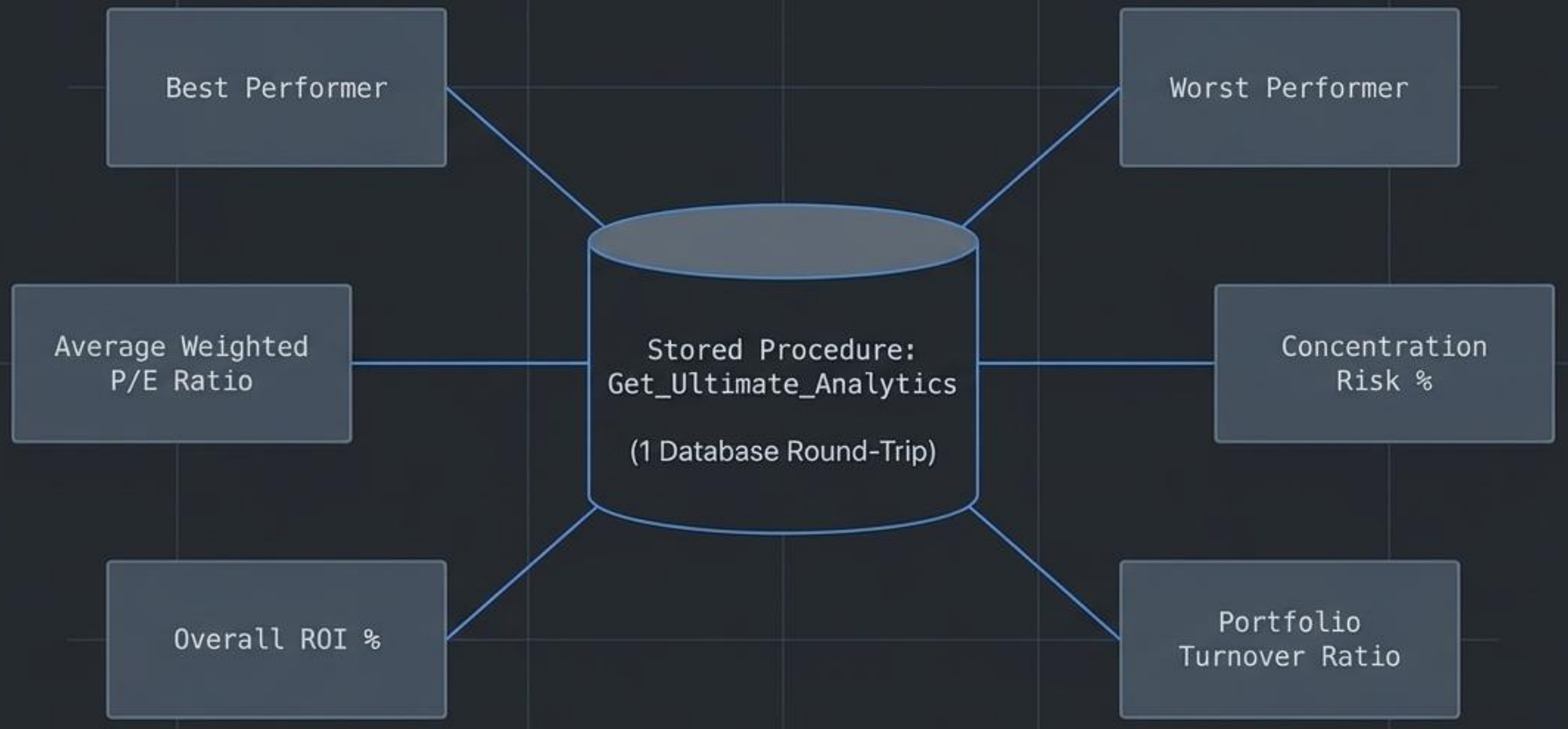


Price Propagation via Autonomous Triggers

```
mySQL>

CREATE TRIGGER After_Stock_Price_Update
AFTER UPDATE ON Stocks
FOR EACH ROW...
```

The Analytics Engine: Single-Trip Computation



Eliminates application-layer loops and minimizes JDBC network payload.

Advanced Query Mechanics

Running P&L
Window Functions

```
SUM(CASE WHEN type = 'BUY'... END) OVER
  (PARTITION BY user_id
   ORDER BY date)
```

Concentration Risk
Aggregate Proportions

```
ROUND(MAX(current_value) / SUM(current_value) * 100, 2)
AS concentration_pct
```

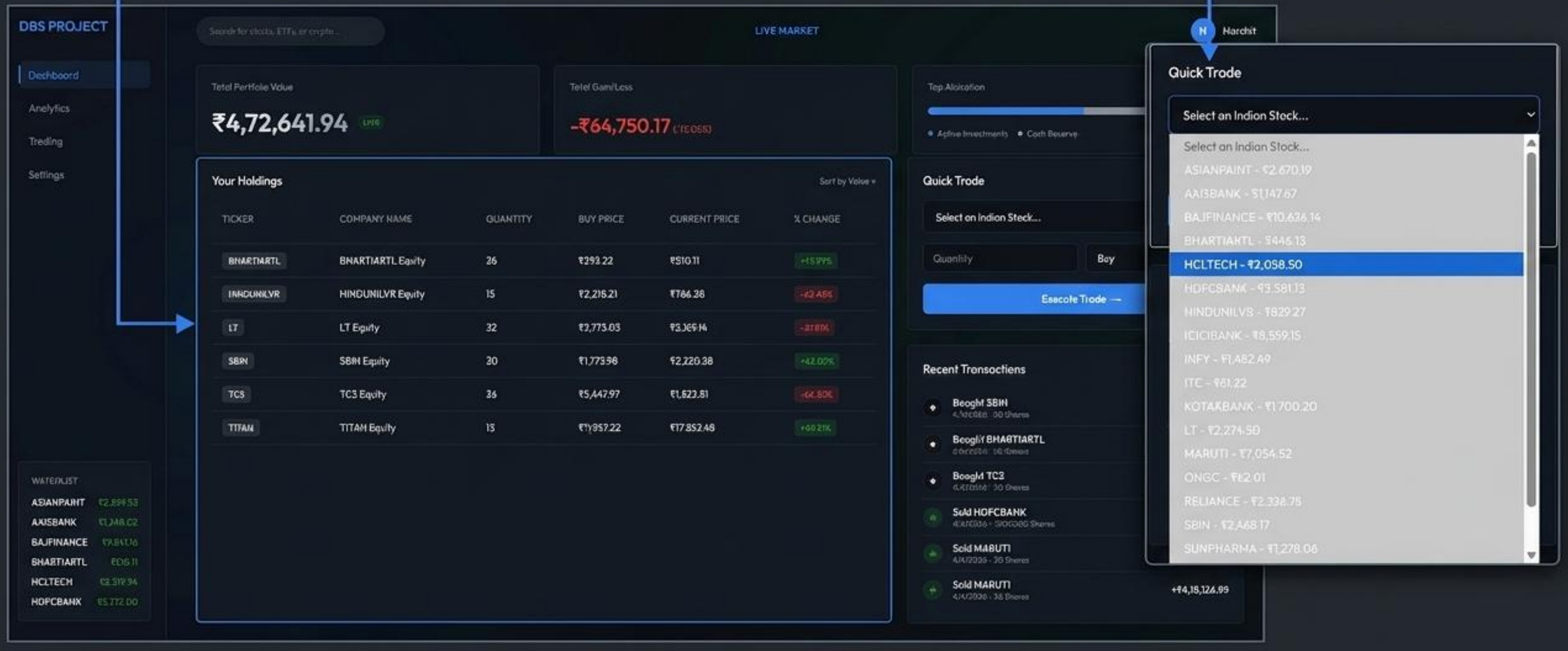
Portfolio P/E Ratio
Value-Weighted Math

```
SUM(s.pe_ratio * p.current_value) /
NULLIF(SUM(p.current_value), 0)
```

Dashboard & Execution Interface

Live P&L Indicators
(Driven by MySQL Triggers)

Asynchronous REST
Trade Execution



Real-Time Institutional Metrics

JSON Payload Delivery

- All 6 metrics fetched asynchronously
- Rendered natively via ES6+ Vanilla JS without frontend calculation overhead.

System Verdict & Validations

[SUCCESS] Schema-First Architecture Validated

Backend successfully drives the frontend; middle tier remains completely stateless.

[SUCCESS] Zero Application-Side Inconsistencies

Trigger automation guarantees portfolio state mathematically matches the transaction ledger.

[SUCCESS] Functional Requirements Passed

All 8 mandated FRs (Registration, Master Data, Triggers, Analytics, REST, SPA) successfully implemented and tested.

Limitations & Future Roadmap

Current State V1

- Simulated stock prices (Manual UPDATEs)
- Singleton JDBC connection bottleneck
- Unauthenticated REST endpoints

Production V2

- Live Alpha Vantage/NSE WebSockets
- HikariCP connection pooling
- JWT authentication & session management
- React/TypeScript migration & Cloud VM deployment